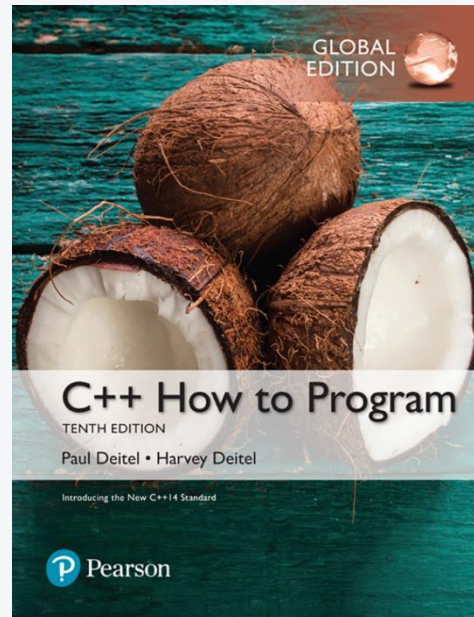


برنامه‌نویسی به زبان C++

مدرس: مریم خراشادیزاده



- **C++ How to Program**
 - Paul Deitel, Harvey Deitel



فهرست مطالب

- فصل اول : مقدمات زبان C++
- فصل دوم : ساختار های تصمیم گیری و تکرار
- فصل سوم: آرایه ها و رشته ها
- فصل چهارم: توابع
- فصل پنجم: ساختارها و اشاره گر ها



فصل اول:

مقدمات C++



زبانهای برنامه‌نویسی

Programming languages



- کامپیوترها ماشینهایی هستند که توانایی انجام محاسبات و تصمیم‌گیریهای منطقی را میلیونها بار سریعتر از انسانها دارند؛ ولی باید برای استفاده از این توانمندیها مجموعه‌ای از دستورالعملهایی را تعیین کرد که به کامپیوتر بگوید چه کاری انجام دهد.
- کامپیوترها پردازش داده‌ها را تحت کنترل مجموعه‌ای از دستورالعملها که **برنامه کامپیوتری (computer program)** نامیده می‌شوند، انجام می‌دهند.
- هر کامپیوتری مستقیماً قادر به فهم و اجرای دستورالعملهایی است که توسط طراح سخت‌افزار آن تعریف شده است که به آن **زبان ماشین** گفته می‌شود. زبان ماشین دنباله‌ای از ارقام (۰ و ۱) است.
- برای ایجاد برنامه‌ها بعد از شناخت مساله و تحلیل نیازمندیها و طراحی (نوشتن الگوریتم حل مساله)، باید **پیاده‌سازی** انجام شود و طراحی انجام شده تبدیل به دستورات زبان ماشین شود تا قابل اجرا برای کامپیوتر شود.
- در مرحله پیاده‌سازی و نوشتن کد برنامه، معمولاً به جای نوشتن برنامه به زبان ماشین از یک زبان برنامه‌نویسی استفاده می‌شود.
- **زبان برنامه‌نویسی**، یک زبان قراردادی است که به کمک آنها می‌توان دستورات لازم برای حل مسائل مختلف را ایجاد کرد. اما دستورات ایجاد شده مستقیماً قابل اجرا توسط کامپیوتر (CPU) نیست؛ بلکه باید به زبان ماشین تبدیل شود.
- به کسی که وظیفه نوشتن کد برنامه و ایجاد آن را به عهده دارد، برنامه‌نویس (programmer) گفته می‌شود.

زبانهای برنامه‌نویسی

Programming languages



- معمولاً زبانهای برنامه‌نویسی را به سه دسته زیر تقسیم‌بندی می‌کنند:
 - **زبان سطح پایین:** در زبان سطح پایین مثل زبان اسمبلی برای راحتی برنامه‌نویسان به جای استفاده از دنباله ارقام ۰ و ۱ از کلمات مختصر شده انگلیسی استفاده شده است.
 - نزدیک به زبان ماشین

```
load    basepay
add     overpay
store   grosspay
```

- **زبان های سطح بالا:** برای افزایش سرعت توسعه برنامه‌های کامپیوتری، به تدریج زبانهای برنامه نویسی سطح بالاتری ابداع شدند که دستورات پیچیده‌تری را برای سرعت و راحتی بیشتر برنامه‌نویسان دربرمی‌گرفتند مثل Pascal, Java, Python, C#. البته این زبانهای برنامه‌نویسی نیاز به مترجمهایی برای تبدیل برنامه‌ها به زبان ماشین دارند که به آنها کامپایلر (compiler) می‌گویند.
- نزدیک به زبان انسانها

```
grossPay = basePay + overTimePay
```

- **زبان های سطح میانی:** در این زبانها مانند C/C++ ترکیبی از ویژگیهای سطح پایین و سطح بالا وجود دارد. به عبارت دیگر این زبانها مشابه زبانهای سطح پایین کنترل زیادی روی حافظه و سخت‌افزار دارند، در حالی که تا حد زیادی امکانات قابلیت‌های زبانهای سطح بالا را نیز دارد.

زبان برنامه‌نویسی C++

C++ programming language



- زبان C اولین بار در سال ۱۹۷۲ توسط Dennis Ritchie در آزمایشگاه Bell توسعه یافت و بعدها به صورت گسترده برای طراحی نرم‌افزارهای مختلف مورد استفاده قرار گرفت. بعدها زبان C++ که توسعه یافته زبان C است، در اوایل دهه ۱۹۸۰ توسط Bjarne Stroustrup ارائه شد. یکی از مهمترین ویژگیهای زبان C++ نسبت به زبان C افزودن مفاهیم شیء‌گرایی است.
- برنامه‌های C++ متشکل از قسمتهایی به نام توابع (function) و کلاسها (class) هستند که توسط برنامه‌نویسان با استفاده از دستورالعملهای زبان C++ توسعه داده می‌شوند.
- برنامه‌نویسان همچنین می‌توانند از توابع و کلاسهای استاندارد که در کتابخانه‌های استاندارد C++ وجود دارد، استفاده کنند. معمولاً این کتابخانه‌ها توسط طراحان کامپایلر توسعه داده شده‌اند.



محیط توسعه یکپارچه C++

C++ Integrated Development Environment (IDE)



پیاده‌سازی و اجرای یک برنامه کاربردی به زبان C++ با استفاده از یک محیط توسعه یکپارچه C++ شامل شش مرحله است که به ترتیب عبارتند از:

1. **ویرایش (Edit):** این مرحله شامل ایجاد یک فایل از دستورالعمل‌های C++ موردنیاز برای برنامه در یک محیط ویرایشگری است که معمولاً به آن کد منبع (source code) می‌گویند. این فایل معمولاً با پسوندهای `.cpp`، `.cxx`، `.cc`، `.C` و ... بسته به محیط توسعه ذخیره می‌شود.

2. **پیش‌پردازش (Preprocess):** معمولاً بعد از ایجاد فایل منبع توسط برنامه نویس دستور کامپایل داده می‌شود، اما در محیط‌های توسعه C++ قبل از کامپایل برنامه، پیش‌پردازش اجرا می‌شود که دستورالعمل‌هایی تحت نام رهنمودهای پیش‌پردازش (preprocessor directives) را روی کد اعمال می‌کند. این رهنمودها شامل کامپایل سایر فایل‌ها و یا جایگزینی متون است که به تدریج با آنها آشنا می‌شوید.

3. **ترجمه (Compile):** در این مرحله کامپایلر برنامه C++ را به کد زبان ماشین ترجمه می‌کند که معمولاً به آن کد مقصد (object code) می‌گویند.

محیط توسعه یکپارچه C++

C++ Integrated Development Environment (IDE)



4. **پیوند (Link):** معمولاً برنامه‌های C++ شامل ارجاعاتی به توابع و داده‌هایی در جاهای دیگر است مثل کتابخانه‌های استاندارد و یا کتابخانه‌های دیگری که توسط برنامه‌نویسان دیگر ایجاد شده است. بنابراین باید این بخشها به هم پیوند خورده و حفره‌های موجود در برنامه پر شود. اگر برنامه به درستی ترجمه و پیوند شود، برنامه‌ای قابل اجرا ایجاد می‌شود.

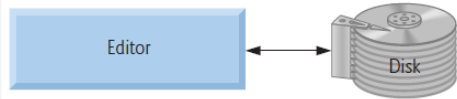
5. **بار شدن (Load):** قبل از اجرای برنامه باید ابتدا فایل اجرایی در حافظه اصلی قرار گیرد. بخشهای دیگر برنامه از کتابخانه‌های مشترک نیز باید در حافظه اصلی قرار گیرد.

6. **اجرا (Execute):** در نهایت کامپیوتر تحت کنترل CPU دستورات قرار گرفته در حافظه اصلی را اجرا می‌کند.

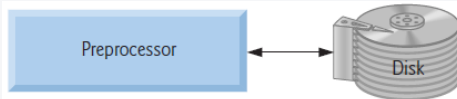
نکته: معمولاً به ندرت در اولین بار همه این مراحل به درستی طی می‌شود؛ بلکه معمولاً خطاهایی رخ می‌دهد که باید با برگشت به مرحله ویرایش دستورات لازم را اصلاح نمود و دوباره مراحل را برای اجرای برنامه طی نمود.

مراحل توسعه برنامه کاربردی C++

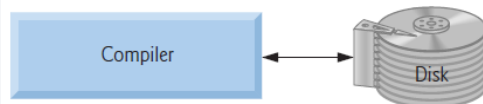
Steps to develop a C++ application



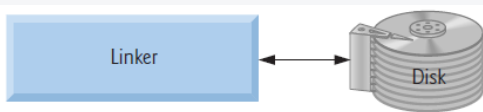
مرحله ۱: برنامه در محیط ویرایشگری ایجاد و روی هارد دیسک ذخیره می‌شود.



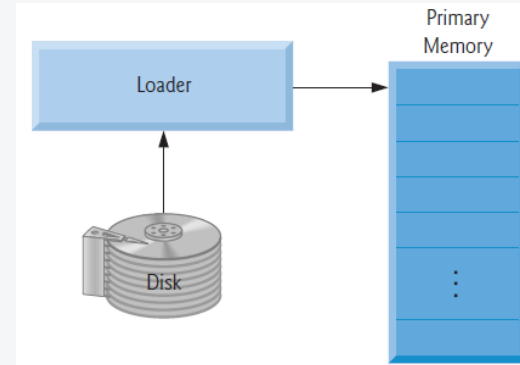
مرحله ۲: پیش‌پردازشگر کد را پردازش می‌کند.



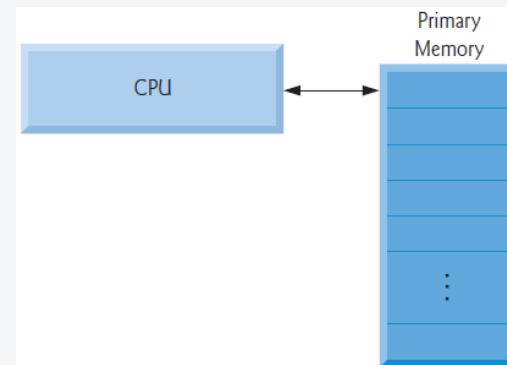
مرحله ۳: کامپایلر برنامه را ترجمه و به عنوان کد مقصد بر روی هارد دیسک ذخیره می‌کند.



مرحله ۴: پیوند دهنده برنامه را با کتابخانه‌های مرتبط پیوند داده و فایل اجرایی را ایجاد و در هارد دیسک ذخیره می‌کند.



مرحله ۵: بارکننده برنامه را در حافظه اصلی قرار می‌دهد.



مرحله ۶: CPU دستورات را به ترتیب اجرا کرده و نتایج را تولید می‌کند.

مثال ۱:



برنامه‌ای که پیغام **Welcome to C++!** را روی صفحه مانیتور نمایش می‌دهد.

```
// Text-printing program
#include <iostream> // allows program to output data to the screen

// function main begins program execution
int main()
{
    std::cout << "Welcome to C++!\n"; // display message
    return 0; // indicate that program ended successfully
} // end function main
```

Welcome to C++!

برنامه‌نویسی در C++

C++ Programming



- یکی از مهمترین اجزای برنامه‌نویسی در زبان C++ توابع هستند. هر برنامه در C++ باید حداقل یک تابع به نام **main()** داشته باشد. مجموعه دستورالعملهای تابع بین { و } قرار می‌گیرد.
- **main** یک **کلمه کلیدی** در زبان C++ به حساب می‌آید. کلمات کلیدی در زبان C++ مجموعه‌ای از کلماتی است که در C++ برای هدف خاصی رزرو شده است.
- اجرای دستورات هر برنامه از تابع **main** شروع می‌شود، حتی اگر اولین تابع پیاده‌سازی شده در برنامه نباشد.
- علامت **//** نشان دهنده توضیحات (comments) در C++ است و برای خوانایی برنامه به کار می‌رود و کامپایلر با مشاهده **//** تا انتهای خط را نادیده می‌گیرد و دستور زبان ماشینی به ازای آن تولید نمی‌کند.
- دستوراتی که قبل از تابع اصلی با **#** شروع می‌شود، رهنمودهای پیش‌پردازش است.
- برای به کار گرفتن دستورات دریافت اطلاعات از صفحه کلید و نمایش اطلاعات در مانیتور باید کتابخانه **iostream** به برنامه افزوده شود.
- هر دستورالعمل در زبان C++ به **;** ختم می‌شود. می‌توان چندین دستورالعمل را در یک خط نوشت. خطوط و فضاهای خالی نیز توسط کامپایلر نادیده گرفته می‌شود.

#include < header file >

int main()

{

// comment

Statement 1;

Statement 2 ;

.

.

Statement n;

return 0 ;

}



مثال ۲:



برنامه‌ای که پیغام **Welcome to C++!** را روی صفحه مانیتور نمایش می‌دهد.

```
// Printing a line of text with multiple statements.  
#include <iostream> // allows program to output data to the screen  
  
// function main begins program execution  
int main()  
{  
    std::cout << "Welcome "; // display message  
    std::cout << "to C++!\n"; // display message  
    return 0; // indicate that program ended successfully  
} // end function main
```

Welcome to C++!

نمایش داده‌ها در خروجی

Displaying Data in the Output



- برای نمایش داده‌ها بر روی صفحه مانیتور از **cout** که به دنبال آن عملگر درج در جریان (**stream** insertion operator) یعنی **<<** وجود دارد، استفاده می‌شود. بایستی توجه داشت که **<<** دو کاراکتر **<** پشت سر هم است.
- هنگام اجرای این دستورالعمل جریان داده‌ها به سمت شی خروجی استاندارد (**std::cout**) که معمولاً به صفحه نمایش متصل است، ارسال می‌شود.
- **std** نوشته شده قبل از **cout** نشان دهنده این است که نام استفاده شده (**cout**) متعلق به فضای نامی **std** است و این فضای نامی با ضمیمه کردن کتابخانه **iostream** به برنامه معرفی شده است.

\n	Newline
\t	Horizontal Tab
\v	Vertical Tab
\b	Backspace
\a	Beep sound
\"	Double quote
\'	Single quote
\?	Question mark
\\	Back slash

- در زبان **C++** بعضی از کاراکترهای خاص برای شکل دهی به خروجی استفاده می‌شود. تعدادی از این کاراکترهای مخصوص به همراه کاربرد آنها در جدول روبرو آورده شده است .

مثال ۳:



برنامه‌ای که پیغام **Welcome to C++!** را روی صفحه مانیتور نمایش می‌دهد.

```
// Printing multiple lines of text with a single statement.  
#include <iostream> // allows program to output data to the screen  
  
// function main begins program execution  
int main()  
{  
    std::cout << "Welcome\n to \n\nC++!\n "; // display message  
    return 0; // indicate that program ended successfully  
} // end function main
```

```
Welcome  
to  
  
C++!
```



متغیر، مکانی در حافظه اصلی کامپیوتر می باشد که در آن می توان یک مقدار را ذخیره کرد و در برنامه از آن استفاده نمود. هر متغیری در C++، نام، نوع، اندازه و مقدار دارد.

اعلان متغیرها (Variable Declarations)

❖ در زبان C++، قبل از آنکه در برنامه به متغیرها مقداری تخصیص داده شود و از آنها استفاده گردد، بایستی آنها را در برنامه اعلان نمود. اعلان متغیرها به شکل زیر است.

Data type Variable_Name;

❖ اعلان متغیر در هر جایی از برنامه می تواند باشد؛ ولی حتما باید قبل از اولین استفاده از متغیر باشد. به کار بردن متغیر قبل از اعلان، خطا در زمان کامپایل ایجاد می کند.

❖ نوع داده های `int`، `float`، `double`، `char` و `bool` انواع داده اصلی در C++ هستند.

انواع داده‌های اصلی در C++

Fundamental data types in C++



نوع داده	محدوده مقادیر (signed)	محدوده مقادیر (unsigned)	حافظه لازم
short (int)	–32,768 تا 32,767	0 تا 65,535	۲ بایت
int	–2,147,483,648 تا 2,147,483,647	0 تا 4,294,967,295	۴ بایت (۲ بایت)
long (int)	–2147483648 تا 2147483647	0 تا 4,294,967,295	۴ بایت
long long (int)	–9,223,372,036,854,775,808 تا 9,223,372,036,854,775,807	0 تا 18,446,744,073,709,551,615	۸ بایت
Bool	true یا false	-	۱ بایت
char	–128 تا 127	0 تا 255	۱ بایت
float	1.2e-38 تا 3.4e38 (۷ رقم دقت اعشار)	-	۴ بایت
double	2.2e-308 تا 1.8e308 (۱۵ رقم دقت اعشار)	-	۸ بایت
long double	2.2e-308 تا 1.8e308	-	۸ بایت (۱۶ بایت)

Identifiers



- شناسه‌ها عباراتی هستند که برنامه‌نویسان برای نامگذاری عناصر برنامه استفاده می‌کنند مثل متغیرها، ثابتها، توابع و...
 - بهتر است برای خوانایی بیشتر برنامه از نامهای معنادار برای اجزای برنامه استفاده کرد.
 - برای نامگذاری شناسه‌ها باید به نکات زیر توجه داشت:
- (۱) در نامگذاری شناسه‌ها از حروف الفباء، ارقام و خط زیر (underscore) استفاده می‌شود و شناسه نمی‌تواند با ارقام شروع شود.
 - (۲) حروف کوچک و بزرگ در نامگذاری شناسه‌ها متفاوت می‌باشند.

بنابراین xy ، xY ، XY ، Xy چهار شناسه متفاوت از نظر C++ می‌باشد.

- (۳) طول نام یک شناسه معمولاً محدودیتی ندارد؛ ولی در برخی از محیط‌ها محدودیتهایی برای طول نام در نظر گرفته می‌شود. برای اطمینان از این موضوع بهتر است طول نام شناسه‌ها **۳۱** کاراکتر یا کمتر باشد.
- (۴) برای نامگذاری شناسه‌ها از کلمات کلیدی نبایستی استفاده نمود. در زیر بعضی از کلمات کلیدی آورده شده است.



and	sizeof	then	xor	template
float	false	friend	while	continue
extern	private	switch	default	const
delete	typedef	if	this	virtual

مثال ۴:



برنامه‌ای که دو عدد صحیح از ورودی دریافت می‌کند و مجموع دو عدد را محاسبه و چاپ می‌کند.

```
#include <iostream> // allows program to output data to the screen
// function main begins program execution
int main()
{
    // variable declarations
    int number1; // first variable
    int number2; // second variable
    int sum;      // third variable

    std::cout << "Enter first integer: "; // display message
    std::cin >> number1; // read first integer from user into number1

    std::cout << "Enter second integer: "; // display message
    std::cin >> number2; // read second integer from user into number2

    sum = number1 + number2; // add the numbers; store result in sum

    std::cout << "Sum is " << sum << std::endl ; // display sum
    return 0; // indicate that program ended successfully
} // end function main
```

```
Enter first integer: 45
Enter second integer: 72
Sum is 117
```


دریافت داده از ورودی

Reading Data from Input



- برای دریافت مقادیر برای متغیرها در ضمن اجرای برنامه از صفحه کلید، می توان از **cin** که به دنبال آن عملگر استخراج از جریان (stream extraction operator) یعنی **>>** قید شده باشد، استفاده نمود. بایستی توجه داشت که **>>** دو کاراکتر **>** پشت سر هم است.
- هنگام اجرای این دستورالعمل جریان داده از شی ورودی استاندارد (**std::cin**) که معمولاً به صفحه کلید متصل است، دریافت می شود.
- همان طور که قبلاً گفتیم **std** نوشته شده قبل از **cin** نشان دهنده این است که نام استفاده شده (**cin**) متعلق به فضای نامی **std** است و این فضای نامی با ضمیمه کردن کتابخانه **iostream** به برنامه معرفی شده است.

```
int x;  
cout << "Enter a number: " ;  
cin >> x;
```

```
char L1, L2;  
cin >> L1 >> L2;  
cout << L1 << L2;
```

```
int x = 10, y = 20;  
cout << "x + y =" << x + y;
```



Assignment operator



- نماد = برای انتساب در زبان C++ استفاده می شود که معادل ← در الگوریتم نویسی است.
- عملگر انتساب «=» باعث می گردد مقدار عبارت در طرف راست این عملگر ارزیابی شده و در متغیر طرف چپ آن قرار گیرد.

مثال :

`x = 35;`

`x = a + b;`

`x = y = z = 26;`

از عملگرهای انتساب
چندگانه نیز می توان
استفاده نمود. که مقدار
سه متغیر Z و Y و X برابر
با 26 می شود.

عملگرهای محاسباتی

Arithmetic operators



در C++ پنج عملگر محاسباتی وجود دارد که عبارتند از :

جمع	+
تفریق	-
ضرب	*
تقسیم	/
باقیمانده	%

- ✓ این عملگرها دو تایی می باشند، زیرا روی دو عملوند عمل می نمایند.
- ✓ عملگرهای + و - را می توان به عنوان عملگرهای یکانی نیز در نظر گرفت.

عملگرهای محاسباتی

Arithmetic operators



- در زبان C++ در حالتی که هر دو عملوند عملگرهای $+$ ، $-$ ، $*$ ، $/$ ، $\%$ از نوع صحیح باشد، نتیجه عمل از نوع صحیح می باشد.

نتیجه	عبارت
7	$5 + 2$
10	$5 * 2$
3	$5 - 2$
1	$5 \% 2$
2	$5 / 2$

عملگرهای محاسباتی

Arithmetic operators



- در زبان C++ در صورتی که یکی از عملوند عملگرهای $\%$ ، $/$ ، $*$ ، $+$ ، $-$ از نوع اعشاری باشد، نتیجه عمل از نوع اعشاری می باشد.

عبارت	نتیجه
$5.5 + 2$	7.5
$5 * 2.1$	10.5
$5.0 / 2$	2.5
$5.3 - 2$	3.3
$5.0 / 2.0$	2.5

عملگرهای افزایش و کاهش

Increment/Decrement operators



- عملگر افزایش «++» باعث می شود مقدار عملوندش یک واحد افزایش یابد.
- عملگر کاهش «--» باعث می شود مقدار عملوندش یک واحد کاهش یابد.
- از عملگرهای ++ و -- می توان به دو صورت پیشوندی و پسوندی استفاده نمود.

مثال :

`++x;`

`x++;`

`x = x+1;`

`--y;`

`y--;`

`y = y-1;`

- ✓ در این سه دستورالعمل به **x** یک واحد اضافه و از **y** یک واحد کم می شود.
- ✓ چون عملگرهای ++ و -- فقط روی یک عملوند اثر دارند، این دو عملگر نیز جزء عملگرهای یکانی می باشند.

عملگرهای افزایش و کاهش

Increment/Decrement operators



در دستورالعمل‌های ترکیبی عملگر پیشوندی قبل از انتساب ارزیابی می‌شود و عملگر پسوندی بعد از انتساب ارزیابی می‌شود.

مثال :

```
int x = 5;  
y = ++x * 2;
```

پس از اجرای دستورالعمل‌های فوق:

```
y = 12  
x = 6
```

```
int x = 5;  
y = x++ * 2;
```

پس از اجرای دستورالعمل‌های فوق:

```
y = 10  
x = 6
```


sizeof operator



sizeof از عملگرهای یکانی می باشد و مشخص کننده تعداد بایتهایی است که یک نوع داده اشغال می کند.

مثال :

```
int x;
```

```
cout << sizeof x ;
```

مقدار ۴ نمایش داده می شود .

```
cout << sizeof(double) ;
```

مقدار ۸ نمایش داده می شود.

عملگرهای انتساب ترکیبی

Compound assignment operators



برای ساده‌تر نوشتن عبارتها در C++ می‌توان از عملگرهای ترکیبی استفاده نمود.

عملگر	مثال	معادل
<code>+=</code>	<code>x += y</code>	<code>x = x + y</code>
<code>-=</code>	<code>x -= y</code>	<code>x = x - y</code>
<code>*=</code>	<code>x *= y</code>	<code>x = x * y</code>
<code>/=</code>	<code>x /= y</code>	<code>x = x / y</code>
<code>%=</code>	<code>x %= y</code>	<code>x = x % y</code>

مثال

```
int x = 5, y = 10;  
x += y;  
y *= 20;
```

```
x = 15  
y = 200
```

پس از اجرای دستورات عملیهای فوق:

عملگرهای رابطه‌ای

Rational operators



این عملگرها بیشتر در شرطها مورد استفاده قرار می‌گیرند.

نام	عملگر
کوچکتر	<
کوچکتر مساوی	<=
بزرگتر	>
بزرگتر مساوی	>=
مساوی	==
نامساوی	!=

عملگرهای منطقی

Logical operators



این عملگرها بیشتر در عبارتهای منطقی مورد استفاده قرار می‌گیرند. با استفاده از عملگرهای منطقی می‌توان شرطهای ترکیبی ایجاد کرد.

نام	عملگر
and	&&
or	
not	!

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

a	b	a b
true	true	true
true	false	true
false	true	true
false	false	false

a	!a
true	false
false	true

عملگرهای بیتی

Bitwise operators



در C++ می توان عملیاتی روی بیت های داده انجام داد.

نام	عملگر
bitwise AND	&
bitwise OR	
bitwise XOR	^
left shift	<<
right shift	>>
bitwise complement	~
bitwise assignment operators	&=, =, ^=, <<=, >>=

تقدم عملگرها در C++

operator precedence in C++



- ارزیابی مقدار یک عبارت براساس جدول اولویت عملگرها انجام می‌گردد. جدول روبه‌رو اولویت عملگرها به ترتیب از بیشترین اولویت به کمترین اولویت نشان داده شده است.

Operator	Description	Associativity
<code>()</code> <code>[]</code> <code>.</code> <code>-></code> <code>++</code> <code>--</code>	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
<code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>!</code> <code>~</code> <code>(type)</code> <code>*</code> <code>&</code> <code>sizeof</code>	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
<code>*</code> <code>/</code> <code>%</code>	Multiplication, division and modulus	left to right
<code>+</code> <code>-</code>	Addition and subtraction	left to right
<code><<</code> <code>>></code>	Bitwise left shift and right shift	left to right
<code><</code> <code><=</code> <code>></code> <code>>=</code>	relational less than/less than equal to relational greater than/greater than or equal to	left to right
<code>==</code> <code>!=</code>	Relational equal to or not equal to	left to right
<code>&&</code>	Bitwise AND	left to right
<code>^</code>	Bitwise exclusive OR	left to right
<code> </code>	Bitwise inclusive OR	left to right
<code>&&</code>	Logical AND	left to right
<code> </code>	Logical OR	left to right
<code>?:</code>	Ternary operator	right to left
<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code>^=</code> <code> =</code> <code><<=</code> <code>>>=</code>	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
<code>,</code>	comma operator	left to right

تقدم عملگرها در C++

operator precedence in C++



$$(5+2) * (6+2*2)/2$$

با توجه به جدول اولویت عملگرها داریم که

$$7 * (6+2*2)/2$$

$$7 * (6+4)/2$$

$$7 * 10 / 2$$

$$70 / 2$$

$$35$$



توضیحات در برنامه باعث خوانائی بیشتر و درک بهتر برنامه می شود. بنابراین توصیه بر آن است که حتی الامکان در برنامه ها از توضیحات استفاده نمائیم. در C++، توضیحات به دو صورت انجام می گیرد.

حالت اول: این نوع توضیح بوسیله // انجام می شود. کامپایلر هر چیزی را که بعد از // قرار داده شود تا انتهای آن خط نادیده می گیرد.

مثال :

```
c = a + b;    //c is equal to sum of a and b
```

حالت دوم: توضیح نوع دوم با /* شروع شده و به */ ختم می شود. کامپایلر هر چیزی که بین /* و */ قرار گیرد، را نادیده می گیرد.

مثال :

```
/* this is a program to calculate sum of  
n integer numbers */
```




Constants

- ثابت ها داده‌هایی هستند که مقدار آنها در طول برنامه تغییر نمی کند.
- دو روش برای اعلان ثابت وجود دارد:

الف) استفاده از کلمه کلیدی **const**

- در این حالت مشابه اعلان متغیر بعد از کلمه کلیدی **const** نوع داده آورده می‌شود و سپس نام ثابت که از قواعد نامگذاری شناسه‌ها پیروی می‌کند.
- ثابت ها حتما باید مقدار دهی اولیه شوند و اگر مقدار دهی آنها فراموش شود، خطا به وجود می‌آید.
- بعد از این که به ثابت ها مقدار اولیه اختصاص داده شد هرگز در زمان اجرای برنامه نمی توان آن را تغییر داد.

const Data type constant_name = value;

ب) استفاده از رهنمود پیش پردازش **define**

- در این روش مشابه رهنمودهای پیش پردازشی از علامت **#** استفاده می‌شود.
- این رهنمودها پیش از بدنه **main** قرار می‌گیرد.
- رهنمود پیش پردازشی در انتها نیازی به ; ندارد.

#define constant_name value

مثال ۵:



برنامه‌ای بنویسید که شعاع دایره را از ورودی دریافت کند و محیط و مساحت دایره را محاسبه کند.

```
#include <iostream>
// #define PI 3.14

using namespace std;

int main()
{
    const double PI = 3.14;
    int radius;
    double perimeter, area;

    cout << "Enter radius: ";
    cin >> radius;

    // PI = 3.1415;          Error
    perimeter = 2 * PI * radius;
    area = PI * radius * radius;

    cout << "perimeter is " << perimeter << "\t area is " << area ;
    return 0;
}
```

با سپاس از توجه شما

